

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Computer Vision with OpenCV

CODE: DSA0201

Name: N Yugesh

Reg No:192424329

CO5: Apply computer vision techniques to real-world applications such as face recognition, tracking, medical imaging, and autonomous systems. (BL3)

Constraint-Based Problem Solving

Code of Conduct:

I N Yugesh(192424329) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Constraint-Based Problem Solving

1. Secure Access System — Face Recognition in Low-Light

Scenario: *A company wants to deploy a secure access system using face recognition in a low-light environment.*

Constraints:

- A. Limited computational resources (embedded device)
- B. Faces may appear at different orientations ($\pm 30^\circ$)
- C. Real-time recognition required (< 200 ms per face)

Q1. How would you adapt Eigenface-based recognition to handle varying illumination and orientation?

Eigenface (PCA-based) recognition operates directly on pixel intensities, so illumination and pose shifts move a face far in pixel space and can dominate the identity-related variance the eigenspace is meant to capture. Practical adaptations:

- Illumination normalization before projection: apply histogram equalization, gamma correction, homomorphic filtering, or a Retinex-based method to strip out lighting variation before the image is projected onto the eigenspace, so PCA captures identity variance rather than lighting variance.
- Pose normalization / alignment: detect facial landmarks (eyes, nose, mouth) and apply an affine warp to frontalize each face within the $\pm 30^\circ$ range before PCA projection, since Eigenfaces are a holistic, pose-sensitive representation.
- Diverse enrollment/training set: build the eigenspace from training images that already span the expected $\pm 30^\circ$ orientation range and multiple lighting conditions, so the eigenvectors themselves absorb some of that variance rather than treating it as noise.
- View-based (multi-eigenspace) matching: instead of one global eigenspace, maintain separate eigenspaces for a few pose bins (e.g., -30° , -15° , 0° , $+15^\circ$, $+30^\circ$); estimate the probe face's pose first, then match against the nearest pose-specific eigenspace.
- Illumination-robust distance metric: use normalized correlation or a whitened/Mahalanobis distance in eigenspace instead of raw Euclidean distance, since Euclidean distance is more easily inflated by illumination-driven magnitude changes.

Q2. Suggest preprocessing steps to improve recognition under low-light conditions.

- Contrast enhancement via CLAHE (Contrast Limited Adaptive Histogram Equalization) to lift local contrast in dark regions without over-amplifying noise the way global histogram equalization can.
- Gamma correction to brighten shadow detail before feature extraction.
- Denoising (e.g., bilateral filtering) applied after contrast enhancement, since low-light sensor noise is amplified by any contrast-boosting step.
- Retinex-based illumination normalization to separate the illumination and reflectance components of the image, retaining reflectance (identity-relevant) detail.
- Active/near-infrared illumination at the hardware level (IR LED plus an IR-sensitive sensor) is a common practical solution for secure access systems, since it sidesteps the low-visible-light problem entirely rather than trying to computationally recover detail that was never captured.
- Standardized face alignment and cropping after enhancement, so downstream PCA processing works on a consistent, tightly-cropped region rather than variable background pixels.

Q3. Discuss strategies to optimize computation for real-time performance on constrained hardware.

- Limit eigenspace dimensionality: retain only the top-k eigenfaces that capture roughly 90-95% of variance; PCA projection cost scales with k, so a small, well-chosen k keeps per-face matching fast without a meaningful accuracy loss.
- Downsample the face crop before projection (e.g., 64x64 instead of 256x256) to cut per-pixel computation, since Eigenfaces do not need high resolution to capture the dominant modes of facial variance.
- Use a lightweight face detector (a reduced-stage Viola-Jones cascade or a small MobileNet-based detector) ahead of recognition, since detection and recognition together must fit inside the 200 ms budget.
- Precompute and cache the mean face and eigenvector matrix once at startup rather than recomputing them per request.
- Exploit fixed-point/quantized arithmetic and any available hardware accelerator (NPU/DSP) on the embedded SoC instead of full floating-point computation.
- Use a coarse-to-fine matching cascade: a fast match with few components first, escalating to more components only for borderline/ambiguous cases, so most faces are resolved cheaply.
- If pose-bin eigenspaces are used, do a fast pose estimate first so only the relevant eigenspace is searched, instead of testing the probe against every pose bin.

2. Photo Album Organization

Scenario: You are designing an AI system to automatically organize a large personal photo collection by people and events.

Constraints:

- A. Millions of images with inconsistent resolution and quality
- B. Multiple photos per person in different poses
- C. Privacy considerations: all processing must occur locally

Q1. How would you use face recognition and clustering to group photos of the same individual?

Pipeline: face detection → alignment → embedding extraction → unsupervised clustering, so that people can be grouped without any pre-existing identity labels:

- Extract a compact, pose/illumination-robust deep face embedding for every detected face (a lightweight, quantized, or distilled local model, since the system must run fully on-device rather than in the cloud).
- Cluster the embeddings with an algorithm that does not require the number of people to be known in advance — e.g., DBSCAN, hierarchical agglomerative clustering (HAC), or graph-based clustering such as Chinese Whispers, which is commonly used in consumer photo-organizing apps.
- Handle multiple poses per person by aligning/frontalizing faces before embedding extraction (reducing pose-induced embedding variance) and by calibrating the clustering similarity threshold wide enough to keep different poses of one person together, but tight enough to still separate different people.
- Use incremental clustering rather than reclustering the whole library on every import: maintain a running centroid per cluster and assign each new face to the nearest existing cluster if it is within the similarity threshold, otherwise start a new cluster — essential at a “millions of images” scale where full reclustering is too expensive to repeat often.

Q2. Propose a method to detect and separate images with occlusions or partial faces.

- Run facial landmark detection (5- or 68-point) on every detected face; if too few landmarks are found, or key landmarks (both eyes, nose, mouth) are missing or low-confidence, flag the face as occluded/partial.
- Compute a simple occlusion/quality score combining: landmark visibility confidence, whether the detected bounding box is cut off by the image border (a strong signal of a partial face), and a blur/sharpness measure.

- Route faces below the quality threshold into a separate low-confidence bucket, or give them reduced weight during clustering (soft assignment) so they do not distort a person's cluster centroid, while still allowing them to be reconciled once a clearer photo of the same person becomes available.
- Where possible, use an occlusion-aware embedding model — one trained with random occlusion/mask augmentation — so that even a partial face still produces a usable, reasonably well-positioned embedding rather than an unreliable one.

Q3. Discuss trade-offs between accuracy and computational efficiency for local processing.

- Model size: larger embedding networks give more accurate, pose/illumination-robust features but are slower and more memory-hungry — a poor fit for processing millions of images entirely on a phone or PC; smaller, distilled, or quantized models trade some accuracy for the speed and footprint that local-only processing requires.
- Clustering method: exact pairwise clustering scales poorly (roughly quadratic) at millions of images; approximate nearest-neighbor structures (KD-trees, HNSW, locality-sensitive hashing) trade a small amount of clustering recall for the scalability the constraint demands.
- Preprocessing depth: heavier per-image enhancement (super-resolution for low-res photos, aggressive denoising) improves feature quality but adds cost that is multiplied across the whole library; a practical design applies lightweight preprocessing by default and reserves expensive enhancement for low-confidence or borderline cases only.
- Because privacy rules out offloading heavy computation to the cloud, the achievable accuracy is architecturally capped by on-device resources (CPU/NPU) rather than being purely a tuning choice — this makes the trade-off a hard constraint of the scenario, not just an optimization nicety.
- A practical compromise: run a fast, lightweight embedding plus approximate clustering for immediate organization, and schedule an optional heavier local refinement pass during idle/charging time to improve cluster quality without affecting everyday responsiveness.

3. Smart City Surveillance — Multi-Camera Pedestrian Tracking

***Scenario:** A smart city surveillance system must detect moving objects and track pedestrians across multiple cameras.*

Constraints:

- A. Crowded scenes with frequent occlusion
- B. Limited network bandwidth between cameras
- C. Real-time processing requirement

Note: the questions for this scenario were not visible in the shared image (the screenshot appears to end right after the constraints). The three questions below follow the same pattern used in Scenarios 1 and 2 — adapting a technique, proposing a specific method, and discussing a trade-off/optimization — applied to this scenario's constraints.

Q1. Discuss strategies to handle occlusion and reliably re-identify pedestrians in crowded, multi-camera scenes.

- Use detectors suited to dense crowds — anchor-free detectors (e.g., CenterNet-style) or detectors trained with occlusion-aware / repulsion losses that better separate heavily overlapping bounding boxes than standard anchor-based detectors.
- Combine motion-based tracking (Kalman-filter / constant-velocity prediction) with appearance-based person re-identification (Re-ID) embeddings, so that when a pedestrian re-emerges after a short occlusion, the track can be re-linked by appearance similarity rather than relying on position alone.
- Manage tracklets explicitly: allow a track to persist through a short occlusion timeout, and reconcile fragmented tracklets after the fact using Re-ID feature similarity, rather than terminating and recreating identities on every occlusion.

- For cross-camera re-identification specifically, use viewpoint- and illumination-invariant Re-ID embeddings trained across multiple camera domains, combined with spatio-temporal constraints (known camera topology and plausible transition times) to reduce false matches that are common in crowded, multi-camera settings.

Q2. Suggest an approach to minimize inter-camera bandwidth while maintaining tracking continuity.

- Perform detection and feature extraction at the edge, on or near each camera, and transmit only compact metadata — bounding boxes, timestamps, and Re-ID embeddings — rather than streaming raw video to a central server.
- Use event-triggered transmission: send data only when a person enters/exits a camera's field of view or when a cross-camera handoff is likely, instead of continuously streaming data for every frame.
- Compress the transmitted Re-ID embeddings (lower-dimensional or quantized/binary representations) to shrink the per-detection payload while retaining enough discriminative power for cross-camera matching.
- Exploit known camera topology (which cameras are physically adjacent and typical transition times between them) so each camera only needs to query or compare against a relevant subset of neighboring cameras, rather than broadcasting to or comparing against the entire network.

Q3. Discuss strategies to meet real-time processing requirements under crowding and bandwidth constraints.

- Deploy lightweight, edge-optimized detection and tracking models (MobileNet/YOLO-tiny-class detectors, quantized Re-ID networks) directly on or near each camera, avoiding round-trip latency to a central server for every frame.
- Decouple the two workloads by latency tolerance: keep per-frame detection and short-term tracking as the tight real-time loop, while cross-camera Re-ID matching runs slightly asynchronously in short batches, since identity resolution across cameras can tolerate a little more latency than frame-level detection.
- Adapt frame rate/resolution dynamically — process at reduced rate/resolution in quieter periods and increase it in denser, higher-risk zones — to keep the average compute load within the real-time budget.
- Use model optimization techniques (pruning, quantization, knowledge distillation) together with available edge hardware acceleration (embedded GPU/NPU) so that per-camera inference reliably meets frame-rate requirements even with many simultaneous detections in a dense crowd.